

Lecture Unit 4.4

Non-blocking communication

Blocking communication

- `MPI_Send` and `MPI_Recv` are blocking communications
- `MPI_Send` will block until the message is received by the destination process or until it is safe to change the send buffer
- `MPI_Recv` will block until the message is received This behaviour helps write correct programs (avoid overwriting a send buffer before it is safe to do so)
- If send and receive operations are not correctly matched there may be a deadlock

Blocking communication

```
#include <stdio.h>
#include <mpi.h>

int main (int argc, char *argv[]){

    int myRank;
    MPI_Status status;
    const int tag=100;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);

    /* Rank 0 sends a number rank 1 and receives a number from rank 1 */
    if ( myRank ==0 ){

        /* Send the number */
        float number_send=3.141;
        printf("I am process %d sending %f\n", myRank, number_send);
        MPI_Send(&number_send, 1, MPI_FLOAT, 1, tag, MPI_COMM_WORLD);

        /* Receive a number */
        float number_rcv;
        MPI_Recv(&number_rcv, 1, MPI_FLOAT, 1, tag, MPI_COMM_WORLD, &status);
        printf("I am process %d received %f\n", myRank, number_rcv);

    } else if ( myRank==1 ) {

        /* Send the number */
        float number_send=1.234;
        printf("I am process %d sending %f\n", myRank, number_send);
        MPI_Send(&number_send, 1, MPI_FLOAT, 0, tag, MPI_COMM_WORLD);

        /* Receive a number */
        float number_rcv;
        MPI_Recv(&number_rcv, 1, MPI_FLOAT, 0, tag, MPI_COMM_WORLD, &status);
        printf("I am process %d received %f\n", myRank, number_rcv);

    }

    MPI_Finalize();

    return 0;
}
```

- If the sends block then both processes will be stuck at the `MPI_Send` calls
- `MPI_Send` is often buffered for small messages which allows the sends to return
- This program may work correctly or may deadlock
- If the message size is increased a previously working program may deadlock

Blocking communication

```
#include <stdio.h>
#include <mpi.h>

int main (int argc, char *argv[]){

    int myRank;
    MPI_Status status;
    const int tag=100;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);

    /* Rank 0 sends a number rank 1 and receives a number from rank 1 */

    if ( myRank ==0 ){

        /* Send the number */
        float number_send=3.141;
        printf("I am process %d sending %f\n", myRank, number_send);
        MPI_Send(&number_send, 1, MPI_FLOAT, 1, tag, MPI_COMM_WORLD);

        /* Receive a number */
        float number_rcv;
        MPI_Recv(&number_rcv, 1, MPI_FLOAT, 1, tag, MPI_COMM_WORLD, &status);
        printf("I am process %d received %f\n", myRank, number_rcv);

    } else if ( myRank ==1 ) {

        /* Receive a number */
        float number_rcv;
        MPI_Recv(&number_rcv, 1, MPI_FLOAT, 0, tag, MPI_COMM_WORLD, &status);
        printf("I am process %d received %f\n", myRank, number_rcv);

        /* Send the number */
        float number_send=1.234;
        printf("I am process %d sending %f\n", myRank, number_send);
        MPI_Send(&number_send, 1, MPI_FLOAT, 0, tag, MPI_COMM_WORLD);

    }

    MPI_Finalize();

    return 0;
}
```

Send and receive match so
this won't deadlock even with
a larger message

Non-blocking send and receive

- `MPI_Send` `MPI_Isend` and `MPI_Irecv` are non-blocking versions of `MPI_Send` and `MPI_Recv`
- Non-blocking communication can avoid deadlocks as we don't have to match sends and receives so carefully
- It can also be more efficient as we can overlap computation and communication (do something else whilst waiting for a message)
- We need to be careful that the program works correctly and only change data in the send buffer once it has been received
- Make a call to `MPI_wait/MPI_waitall` at the point in the program when the communications need to have completed

MPI_Isend

- MPI_Isend: non-blocking send
- `int MPI_Isend(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm, MPI_Request *request)`

buf: send buffer

count: number of items to send

datatype: type of data

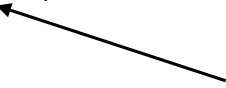
dest: which rank to send to

tag: user defined label for this communication

comm: communicator e.g. MPI_COMM_WORLD

request: handle for this request (output)

Extra argument



MPI_Irecv

- MPI_Irecv: non-blocking receive
- `int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Request *request)` ← Instead of status

`buf`: receive buffer

`count`: number of items to receive

`datatype`: type of data

`source`: where to receive from

`tag`: user defined label for this communication

`comm`: communicator e.g. `MPI_COMM_WORLD`

`request`: handle for this request (output)

MPI_Wait

- `MPI_Wait`: wait for a request to complete
- `int MPI_Wait(MPI_Request *request, MPI_Status *status)`

`request`: handle for the request to wait for (input)

`status`: status of the request (output)

MPI_Waitall

- `MPI_Waitall`: wait for all requests to complete
- `int MPI_Waitall(int count, MPI_Request array_of_requests[], MPI_Status array_of_statuses[])`

`count`: number of requests to wait for

`request`: array of request handles to wait for

`status`: array of statuses of the requests

Collectives

- The collectives in unit 4.3 are blocking calls
- All processes in the specified communicator must reach the collective communication or there will be a deadlock
- A blocking collective synchronises all the processes in the communicator
- Non-blocking collectives were introduced in version 3 of the MPI standard

Unit summary

- Blocking communication may block until the message is received
- This can cause a deadlock if sends and receives don't match
- Non-blocking communication can help avoid deadlocks
- Non-blocking communication also allows computation and communication to be overlapped
- The non-blocking send and receive calls are `MPI_Isend` and `MPI_Irecv`
- Make a call to `MPI_Wait` or `MPI_Waitall` to ensure requests have completed